Emergency Dispatch Priority and Coordination System

Final Project Report

ENSE707 Software Quality Assurance

Leo Cao (23211788), Fateh Bhular (23217987), Shawn Lee (23204035)

Auckland University of Technology

Table of Contents

1. Project Completion and Final Implementation....4
1.1 Project Overview....4
1.2 Final Scope and Changes Since Assessment 1 (What did we change?)....4
1.3 Final System Architecture and Design....6
1.4 Final Implemented Functionality....8
1.5 Final Prototype Evidence (Assignment 2 system)....10
1.6 Remaining Implementation Limitations....13
2. Requirements Verification and Traceability....14
2.1 Requirements Review and Changes Since Assessment 1....14
2.2 Final Requirements Traceability Matrix....16
2.3 Partially Met, Unverified and Out of scope Requirements....19
3. Test Execution and Final Test Results....20
3.1 Final Test Cases....20
3.2 Test Execution Results....22
3.3 Failed Tests and Corrective Actions....23
3.5 Remaining Testing Risks....25
4. Test Automation and Quality Engineering....26
4.1 Automation Selection....26
4.2 Automated Test Implementation....27
4.3 Automated Test Results....28
4.4 Automation Maintainability....28
4.5 Manual and Exploratory Testing....29
5. Continuous Integration and DevOps QA....30
5.1 Development and Integration Workflow....30
5.2 Continuous Integration Pipeline....31
5.3 Quality Gates....32
5.4 Smoke testing....33
5.5 CI Evidence and Limitations....35
6. Defect Management and Root Cause Analysis....37
6.1 Defect Management Framework....37
6.2 Final Defect Register....38
6.3 Root Cause Analysis of Significant Defects....39
6.4 Defect Prevention and Quality Improvement....43
7. Quality Testing....44
7.1 Risk-Based Selection of Quality Areas....44
7.2 Quality Area One: [e.g. Security]....44
7.3 Quality Area Two: [e.g. Performance]....44
7.4 Results and Limitations....44
7.5 Recommended Improvements....44
8. Quality Metrics and Final Release Decision - lao....45
8.1 Final Quality Metrics....45
8.2 Requirements and Acceptance-Criteria Evaluation....45
8.3 Outstanding Defects and Residual Risks....45

8.4 Final Release Decision..... 45
9. AI-Assisted Development and Critical Reflection.....46
9.1 AI-Assisted Activities..... 46
9.2 Representative GitHub Copilot Evidence.....47
9.3 Human Review and Validation.....47
9.4 AI-Related Quality Risks.....47
9.5 Critical Reflection..... 47
10. Quality Improvement and Future Recommendations.....48
10.1 Key Quality Lessons - Leo..... 48
10.2 Improvements to the Development and QA Process - fateh.....48
10.3 Future Testing and Automation Improvements - Leo.....48
10.4 Future Continuous Quality Engineering - Shawn.....48
11. Team Contribution and Professional Reflection.....49

1. Project Completion and Final Implementation

1.1 Project Overview

There is a high volume of calls received by emergency dispatchers every day in New Zealand alone. Sometimes there are too many cases, so even office workers have to go out into the field to keep up with all of them *(1News Reporters, 2026).* This causes stress on the current emergency system, which underlines the need for a new response system that can effectively manage all emergency cases and coordinate response units for these cases. The Emergency Dispatch Priority and Coordination System was developed to improve the management of emergency calls, case handling, prioritisation, department routing, and response-unit coordination. The prototype from phase I allows the dispatcher to create a case and put it through the system, which assigns a priority. The case is then routed to the assigned departments, where they can view the case details on their dashboards with very quick updates. After a unit has finished a case, they can update it on their dashboard, and once all units have signed off, the case closes.



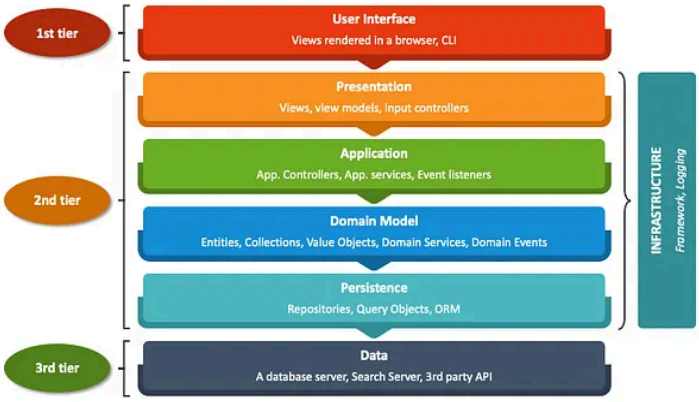
1.2 Final Scope and Changes Since Assessment 1 *(What did we change?)*

The original scope from assignment 1 was to develop an emergency dispatch priority and coordination system prototype that managed emergency cases, case prioritisation, case routing to departments, and response-unit coordination. The scope for the final assignment includes some changes to the features due to new findings, corrections required from the previous scope, as well as new ideas.

Scope Area	Assessment 1 Scope	Assignment 2 Scope	Justification
Case recording	Dispatchers can record caller details, incident type, description, location, severity, and required departments in a form. Submitting this form creates a new case.	**No scope change.** The required inputs remain validated, and the case number and timestamp are generated automatically during case creation.	Case recording provides the information that is used in prioritisation, routing, unit assignment, and testing. The automatic timestamp and case number improve consistency.
Prioritisation	The system automatically determines the case priority using the department-related rules. The dispatcher's selected severity is used when there are no keyword matches.	**Scope changed.** An authorised dispatcher can override the calculated priority by giving a mandatory reason, which is stored in the audit log.	Priority override allows a miscalculated priority to be corrected. The original priority, updated priority, user, reason, and the timestamp should all be recorded for accountability,
Routing	Cases are routed to every department selected by the dispatcher. Multi-department cases appear in all of the selected departments' interfaces.	**No scope change.** Selected departments can view the case on their dashboards. Cases are excluded from unrelated departments' dashboards.	This workflow satisfies the multi-department routing scope from assignment 1. There is no external emergency-service integration in this prototype.
Unit assignment and coordination	The first available unit from each required department will be assigned to the case. If no compatible units are available, the case will be in a queue.	**Scope changed.** Unit assignment assigns the nearest available unit using its stored location data. If no unit is available, the case remains in the queue.	Nearest-unit selection can reduce travel distance for the selected response unit and the expected time of arrival. This is supported by location data that is measurable, defined rules, and repeatable tests.
Case lifecycle, sign-off, and history	Cases use the following states: Open, In Progress, Closed. Each unit signs off individually, and the case only closes when all units have signed off. Dispatchers can search previous cases by caller name, case number, or date.	**No scope change.** The case lifecycle, unit sign-off, and case history are kept the same. Case closes after all required response units have signed off. History remains searchable by the inputs mentioned.	The lifecycle prevents cases from being closed too early on multi-department cases, and keeps all states consistent throughout the case lifecycle. The history supports traceability and access to previous cases.
Role-based GUI	There are separate interfaces and restrictions provided for dispatchers, departments, response units, and the prototype admin role. Department and response unit users are restricted to their assigned scope.	**No scope change.** The role-based interfaces and server-side access controls are kept. The admin is prototype only and is used for testing and demonstration.	Role-specific interfaces reduce the complexity and prevent users from accessing pages they shouldn't be allowed to access. Admin is kept for the prototype for easier testing and demonstration.
Architecture & Technology	This system uses C#, .NET, Razor Pages, HTML/CSS, MSTest, and GitHub with a modular design.	**No scope change.** The structure, repository abstractions, priority strategy, dependency injection, and central dispatch service are kept.	Keeping the existing technology stack avoids migration and regression risks. Explaining the architecture shows how the prototype supports maintainability, extensibility, and testing.
Security and Persistence	The system uses in-memory repositories and fake accounts in text files. Operational data was lost after restarting the program. Role-based authorisation was implemented, but not permanent storage or encryption.	**Scope changed.** Includes permanent database storage, password hashing, encryption at rest, and a persistent audit log. The audit log records important actions related to a case, timestamps, and the responsible user.	Persistent storage prevents important data from being lost after the program restarts. Password hashing and encryption reduce the risk of sensitive data being exposed if the database is accessed directly. Audit logging improves accountability, traceability, and security review.

1.3 Final System Architecture and Design

There are six layers used in our system architecture: Domain, Application, Logic, Infrastructure, Web, and Authentication. This separation in the system means that each part can focus on one responsibility, while still contributing a part to the complete dispatch workflow.



The **domain layer** contains the system's main objects, with each of them controlling its own rules, functionality, and state changes in the system. For example, assigning cases, releasing a unit, or closing a case. The main objects are cases, units, departments, priorities, and case states. The **application layer** is the coordinator of the system and tells parts of the system "what needs to happen". DispatchService is an example in the application layer, where it coordinates the case creation, prioritisation, routing, and case-related tasks. The **logic layer** contains the rules about the system objects that are independent of the interface and database. For example, KeywordSeverityPriority.cs implements the priority logic in the system, where its rules can be changed independently. The **infrastructure layer** contains all of the department, case, notification and other repositories. These have direct access to the database and its chosen operations. The **web layer** uses ASP.NET Core Razor Pages to display the interface and pages of the system. This includes the sign-in page, dispatcher dashboard, department dashboard, and all other system pages. Lastly, the **authentication layer** holds files that are responsible for credential checking and making sure users can only access information that is permitted by their role.

Our system implements **two main design patterns: the repository pattern and the strategy pattern**. The repository design pattern gives an abstraction layer between the business logic and data storage. This gives a consistent way of managing data while hiding the implementation details *(GeeksforGeeks, 2026)*. ICaseRepository, IDepartmentRepository and IDispatchNotifier all use the repository design pattern to separate the workflow from the storage. The strategy design pattern is a behavioural design pattern where a group of similar algorithms are used in a way that makes them interchangeable. This allows the algorithm to vary depending on what the client requires for their task, and enables these changes during runtime without modifying the existing code *(GeeksforGeeks, 2026)*. In our system, the strategy pattern was used for case priority calculation. The IPriorityStrategy interface defines the framework to calculate priority, and KeywordSeverityPriority provides the implementation of that framework.



We used **dependency injection** to connect the different layers and components of our system architecture. All of the repositories, priority strategy, notification service, and the dispatch service are loaded when the application starts. Rather than every class creating its own dependencies created unnecessary load on the system, the components are provided to those classes instead. This reduces the coupling between components while also making testing easier because the repositories and services can be provided for testing instead of the real implementations.

The DispatchService was implemented as the **central service for the main system workflow**. It coordinates the case creation, priority calculation, storage, unit assignment, notifications, unit sign-off, and the reassignment of released units to the cases in queue. The intention of this service was to prevent similar workflows from being implemented in different locations of the system. The pages receive user input and display information, while DispatchService coordinates what should happen to the case and units (backend of the application).

Domain-controlled state was used to reduce the possibility of invalid states during runtime. The states of cases and response units are controlled by the domain objects instead of the user interface. The

Case object controls whether the unit can be assigned, when a unit signs off, and whether the case should be Open, Closed, or In Progress. The Unit object controls its availability and the current case assignment. With domain objects controlling their own states, it prevents invalid states. For example, an unavailable unit still being assigned to another case, or a multi-department case closing when not all units have signed off.

All of the design decisions and patterns implemented improve the quality of the system and make it easier to test and maintain. The **layered architecture helps with testing**, due to all the business logic being separated from the web interface. This allows the business logic to be tested without needing to load the interface, allowing for easier automated testing. The **repository design pattern reduces the coupling** between components because the application layer doesn't worry about how the data is stored. Overall, the design decisions make the system more modular and easier to modify without needing to also modify other unrelated layers. The persistent database, password hashing, and audit logging added improve the reliability, security and traceability of the system. This is because it prevents data loss after the program restarts, protected information is secure, and important actions on cases are recorded for accountability.

1.4 Final Implemented Functionality

The following table summarises the core functionality implemented in the final version of the Emergency Dispatch Priority and Coordination System

Requirement	Final implemented functionality
FR-01 & FR-02	An emergency case is recorded, validated, and created based on caller, incident, location, severity, and department information is validated correctly
FR-03	Department users can only view their units and update unit location and their worker count. The unit availability is controlled by assignment and sign-off.
FR-04	The system calculates priority using department rules. An authorised dispatcher can override the priority result with a mandatory reason

FR-05	The cases appear on every selected department's dashboard and are excluded from unrelated department dashboards.
FR-06	The nearest available response unit is selected using its stored location data. If no unit is available, the case remains waiting until a compatible unit is free.
FR-07	Each department dashboard only displays its relevant cases and includes new routed cases automatically. The page refreshes every 5 seconds.
FR-08	The case follows the defined state rules (Open, In Progress, and Closed) in waiting, assignment, partial sign-off, and completion.
FR-09	Units become unavailable when assigned and available again after signing off. Two requests at the same time cannot give one unit two active assignments.
FR-10	Each unit that is assigned receives a notification containing the case number, incident type, and location. If the notification fails, it should not undo successful case creation or assignment.
FR-11	Units sign off individually; if units are still assigned to a multi-department case, its state should be In Progress. The case closes after all assigned cases have signed off.
FR-12	Dispatchers can search by caller name, case number, or date. Cases that match any of the fields are returned.
FR-13	Important actions include case creation, priority override, routing, assignment, sign-off, and closure. These are stored with the case action, user or system actor, and timestamp. Audit records should remain accessible after the program restarts.
NFR-S02 & NFR-U01	Dispatchers, departments, and response units can access only their allocated pages and scope. Any unauthorized page requests should be denied.
NFR-R03	Cases, units, assignments, notifications, accounts, and audit records should remain accessible after the application has restarted. Stored data should be retrieved without missing or changed fields.
NFR-S01	Sensitive account data should not be stored as readable plaintext in the local database. Authorised operations can encrypt, store, retrieve, and decrypt the data correctly.
NFR-S03	Account passwords should be stored using password hashing. Correct passwords should be accepted, while incorrect passwords and direct hash comparison attempts are rejected.
NFR-P01	Cases should be sent to the relevant department dashboards in under 2 seconds.

1.5 Final Prototype Evidence *(Assignment 2 system)*

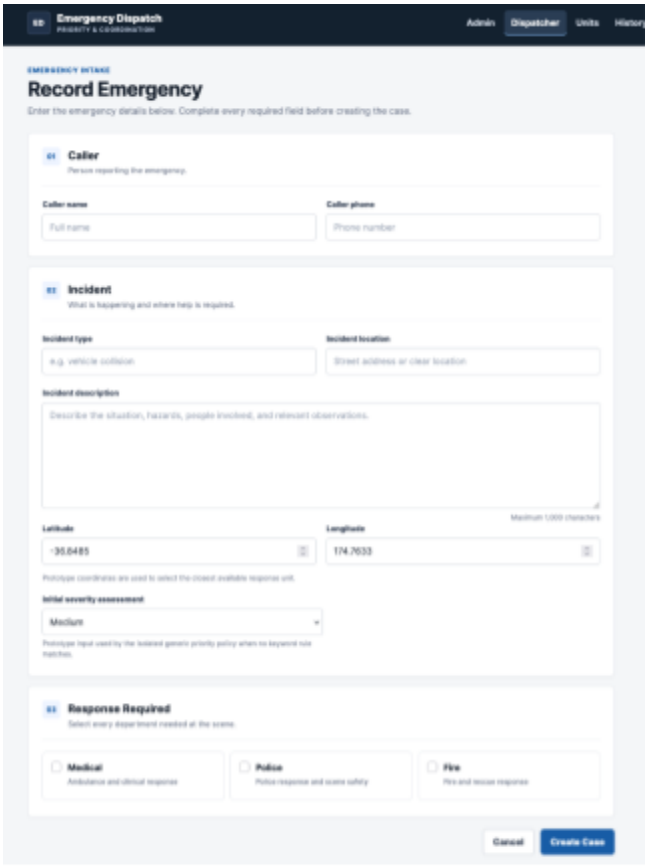


Figure 1.5.1: Dispatcher case-recording form

This is the /Cases/Create page where the caller details, incident details, location, reported severity, and department selection are captured. It provides dispatchers with a simple GUI to enter details rather than the older, confusing page. The location input is sent to the unit-assignment service, where it checks the coordinates of the location and finds the nearest available unit from each selected department. All mandatory information is checked using server-side validation, and the severity is checked using the priority system, which calculates the final priority for the case. The priority can also be overridden with a mandatory explanation of why it is changing. It will then be saved to the audit log, and the case details will be saved on the dispatcher's dashboard (*Figure 1.5.2*).

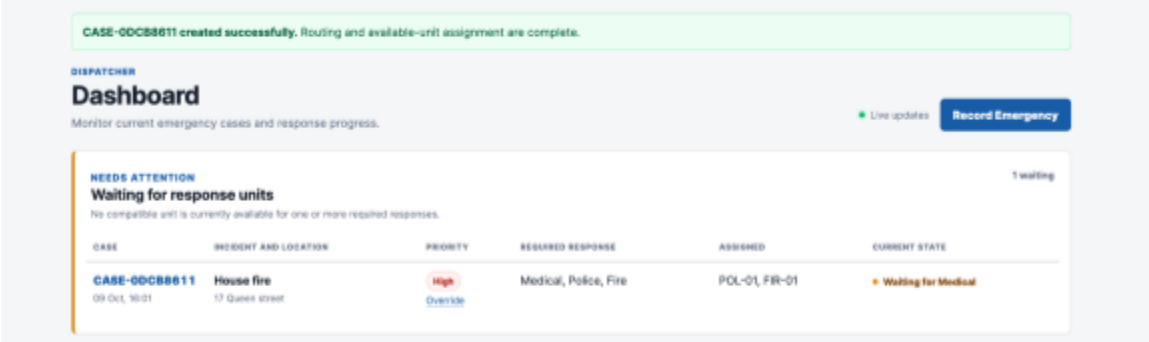


Figure 1.5.2: Case on dispatcher's dashboard

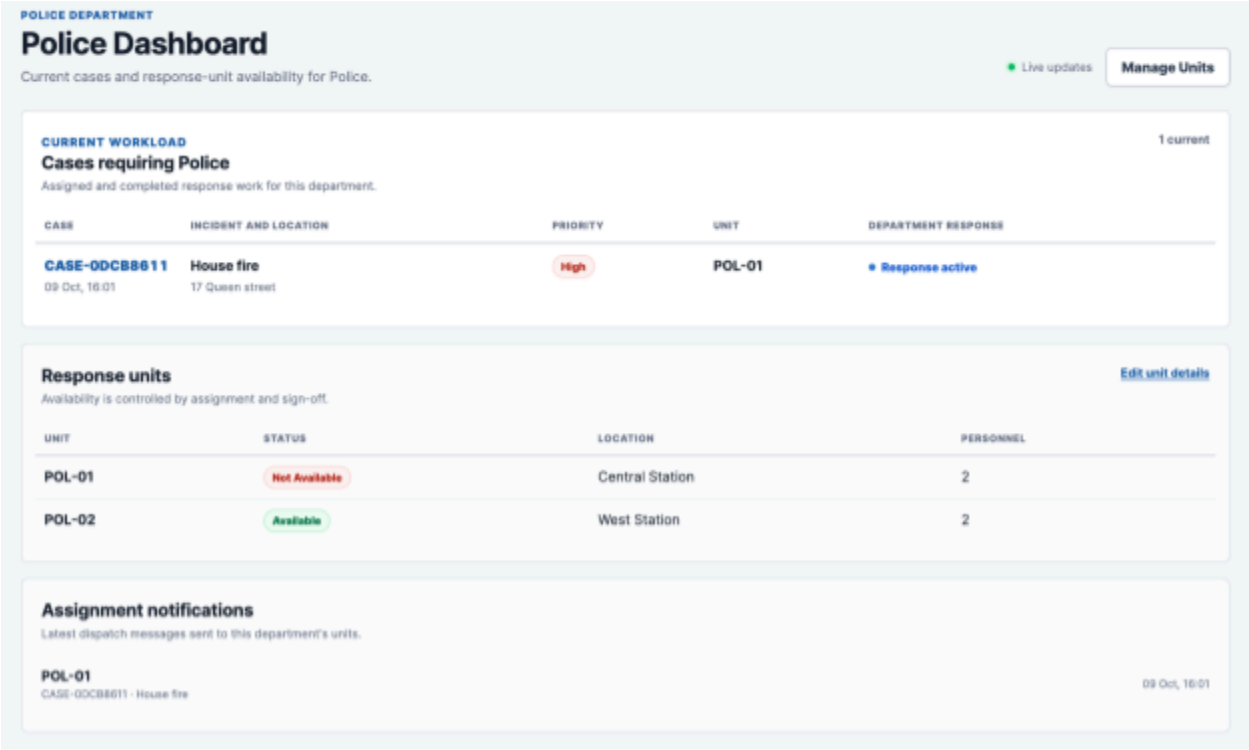


Figure 1.5.3: Department-specific dashboard

Figure 1.5.3 shows the police department's dashboard, where only cases assigned to this department will be displayed. Each dashboard shows the current cases that require police units, the list of all response units along with their availability status, and any notifications from the system. Each of these components is separated for easier viewing, with the goal of users being able to interpret critical information quickly. Dashboards reload every 5 seconds so that any new information is quickly updated, without the user having to manually reload the page, and reduce the chances of missed information.

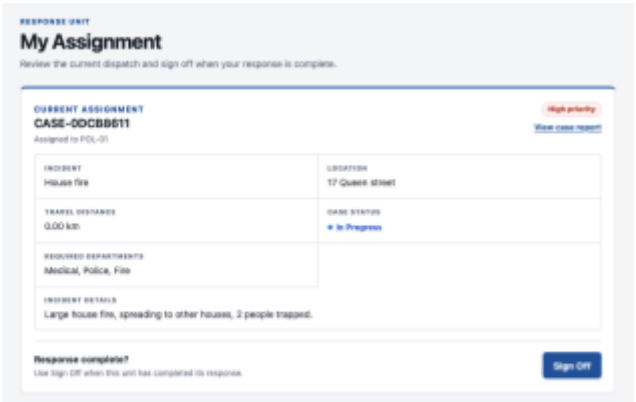


Figure 1.5.4: Response-unit workspace

Figure 1.5.4 shows the interface of the POLICE-1 unit, where they receive the current assignment and its details. The dispatcher's note is added because it provides important information that helps the unit prepare for the case. After the case is handled, the unit can sign off from the case, which only closes when all assigned units have signed off.

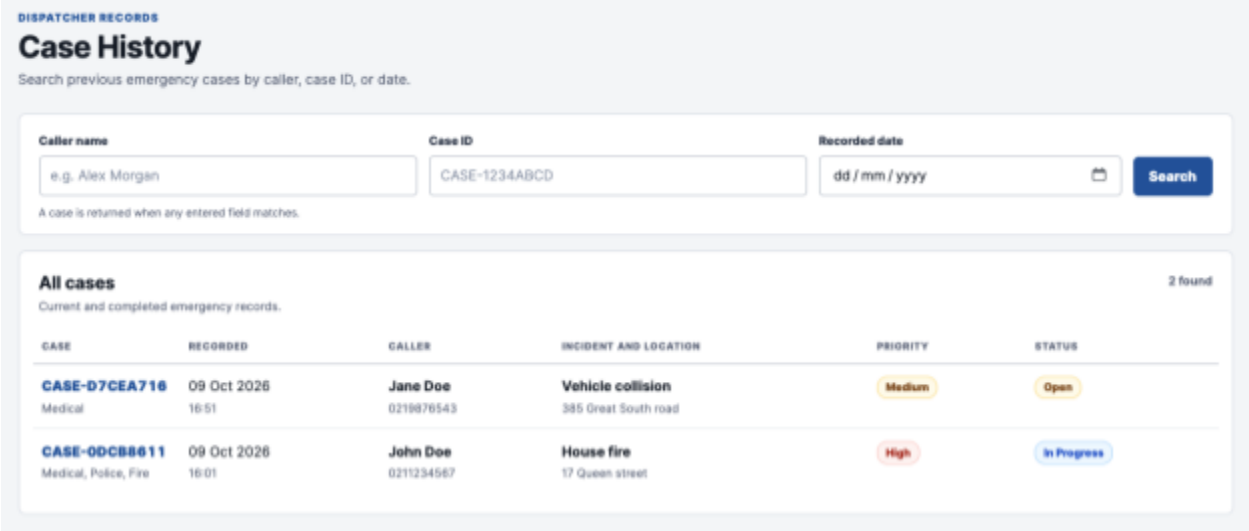


Figure 1.5.5: Case history showing case in queue

When no response units are available, the newly created case goes into a queue, with the status “Open”. Figure 1.5.5 shows the case that was just created with the status in the case history log. In the case history page, the dispatcher can search for cases by caller name, case ID, or the date it was created.



Figure 1.5.6: Audit log of case actions

Figure 1.5.6 shows the audit log of actions taken by different users. Every major action: creation, assignment, override, sign-off, and closure, is saved to this log to track who did what and to help with traceability. If an important event happened and someone had to figure out who performed a certain action, this log helps them find which user did it, making it easier for investigation if it occurs.

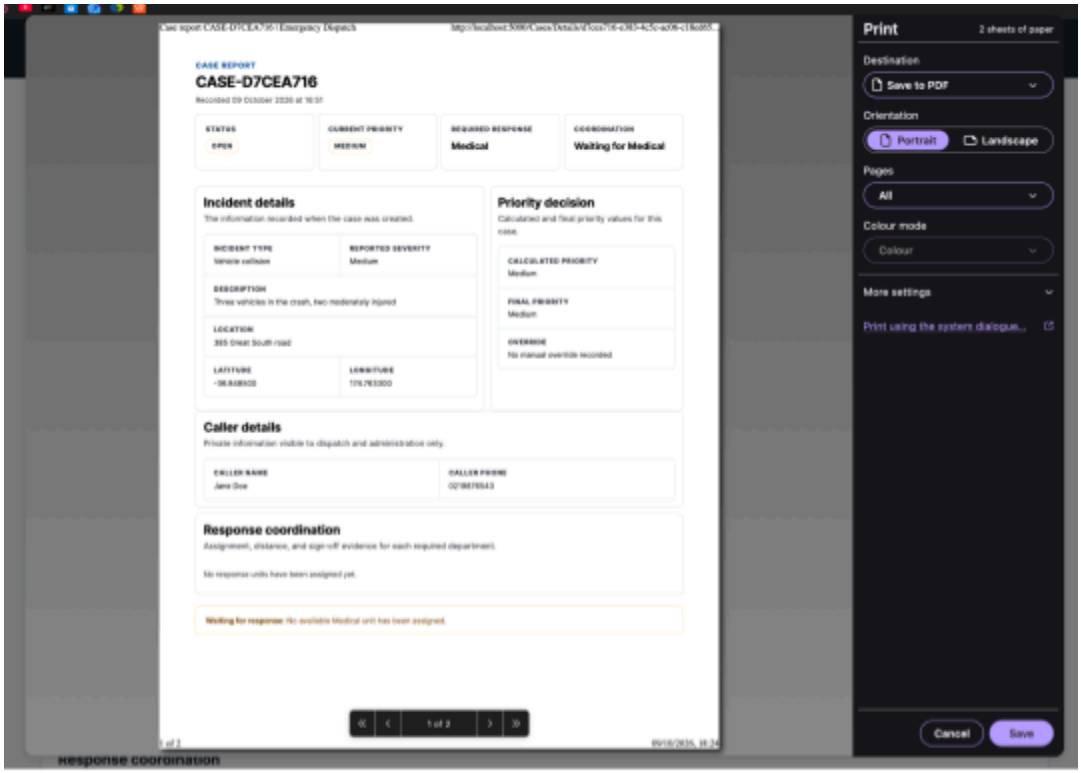


Figure 1.5.7: Printable case report

All cases have a “print report” button which generates a report of the case details and its full lifecycle. It shows the incident details, priority decision including the override details if any, caller details, the response coordination, and finally the case activity (displayed like the audit log, but only actions linking to this case).

1.6 Remaining Implementation Limitations

The final prototype implements all of the important requirements of a real-life emergency dispatch system, but it is still a local hosted prototype. These requirements include case recording, department routing, prioritisation, response-unit assignment, and audit logging. The routing is done locally, so it does not communicate with anything outside of this local hosted prototype, like real life emergency services. The distance calculation required coordinates, and adding a coordinate finder was out of the scope. Therefore, the dispatcher has to enter coordinates from a mock coordinate list, and use that for demos.

Dashboards are updated every five seconds using polling. In the future, we would like to add real-time push updates, which would allow departments to receive cases instantaneously, instead of five seconds later. Another limitation is that the SQLite database and locking have only been tested for this prototype and are not viable for large distributed systems.

As an Emergency Dispatch Priority and Coordination system, the security is not suitable for what it should be for a real-life emergency system. Our prototype still lacks secure database management, authentication methods, and firewalls for system protection, so while it is a functioning prototype, it should not be applied to real-life emergency scenarios or implemented into systems.

2. Requirements Verification and Traceability

2.1 Requirements Review and Changes Since Assessment 1

The refined functional and non functional requirements that were created in assignment one were revisited during the development and testing of the final prototype. The complete list of all requirements from assignment one is provided in **Appendix 5**.

The changes made in the final prototype showed that the original requirements no longer reflected the intended behaviour of the new prototype. As a result, we had to revisit the original functional requirements; making sure it properly described the intended behaviour, and adding new requirements for functionalities introduced in assignment 2. All these updated functional requirements for assignment two are provided in **Appendix 6**, and the upcoming section only mentions the changes that were made. Minor changes such as rewording or clarity were not included in the table below because they did not change the intended behaviour of the requirement.

The table only identifies requirements where the expected system behaviour or project scope was meaningfully changed, removed, or added during Assessment 2. Only the requirements which have been meaningfully changed in expected system behaviour or project scope are identified in the table.

2.1.1 Functional Requirement Changes

ID	Change from Assessment 1	Reason
FR-01	Added coordinates and reported severity, and clarified the required response departments and timestamp.	To support the updated case recording and priority/assignment requirements.
FR-03	Unit availability is now controlled automatically instead of being manually updated by departments.	To prevent inconsistent unit availability states.
FR-04	Added an authorised dispatcher priority override with a required reason.	To allow incorrect calculated priorities to be corrected with accountability.
FR-06	Changed unit assignment from the first available unit to the nearest available unit.	To make unit selection location-based rather than list-based.
FR-08	Removed the unused assigned-unit removal scenario and clarified the three case states.	To match the agreed case lifecycle.
FR-13	**new requirement;** added persistent audit logging for important system actions.	To improve traceability and accountability.

2.1.2 Non-Functional Requirement Changes

ID	Assessment 2 change	Reason
NFR-R03	Expanded persistence to cases, units, assignments, notifications, accounts and audit events across restarts.	To cover all persistent data used by the final system.
NFR-S01	Added AES-256-GCM encryption for selected sensitive stored data.	To protect sensitive data stored by the system.
NFR-S02	Added department/unit scope and denial of direct unauthorised requests.	To strengthen access control beyond the interface.
NFR-S03	**New requirement** for salted PBKDF2 password hashing.	To prevent passwords from being stored in plaintext.

2.2 Final Requirements Traceability Matrix

The final requirement traceability matrix links every requirement defined in the **Appendix 6** document to its implemented functionality, verification evidence, final result, and related defects where applicable. Section 3 gives more details on the testing procedures and execution results. *All the labeled defect IDs will be further elaborated in section 6.*

2.2.1 Functional Requirements Traceability Matrix:

ID	Final implementation	Verification Evidence	Result	Defect
FR-01	Validated Razor form records caller, incident, location, coordinates, severity and selected departments.	TC-01, TC-02 and HTTP workflow	**Passed**	D-06
FR-02	Creates one case with a generated identifier and timestamp. Initial status depends on unit availability.	TC-01, TC-03 and HTTP workflow	**Passed**	–
FR-03	Department users can view and update their units' location, coordinates and personnel count. Availability is controlled by assignment and sign-off.	FR-03 valid/invalid update tests, restart test and role tests	**Passed**	D-06
FR-04	Department-specific keyword rules calculate priority, with reported severity as the fallback. Authorised overrides require a reason.	TC-04 and PriorityOverrideTests	**Passed**	D-06
FR-05	Cases are visible to selected departments and excluded from unrelated department dashboards.	TC-03, TC-05, TC-09, role tests and HTTP tests	**Passed**	–
FR-06	Selects the nearest compatible available unit using Haversine distance. Cases wait when no compatible unit is available.	TC-06–TC-08 and DistanceAssignmentTests	**Passed**	D-03
FR-07	Department dashboards contain only relevant cases and reload every five seconds.	TC-09 and rendered-page HTTP refresh test	**Automated checks passed**	D-06

FR-08	Domain rules control Open, In Progress and Closed case states.	TC-03, TC-07, TC-10 and database lifecycle test	**Passed**	–
FR-09	Assignment makes units unavailable; sign-off releases them; locking prevents duplicate active assignment.	TC-06–TC-10, concurrency test and database lifecycle test	**Passed**	–
FR-10	Assigned units receive persistent case notifications. Notification failure does not undo a successful dispatch.	TC-03, notification-failure test and notification restart test	**Passed**	D-04
FR-11	Units sign off individually, and a case closes after the final required response signs off.	TC-10, database lifecycle test and HTTP workflow	**Passed**	–
FR-12	Dispatcher history supports OR-based search by caller, case number or date.	TC-11 and repository search tests	**Passed**	–
FR-13	Important case actions are stored persistently with actor, time, case and unit information.	AuditLoggingTests, PriorityOverrideTests and database lifecycle test	**Passed**	D-01, D-06

2.2.2 Non-Functional Requirements Traceability Matrix:

ID	Final implementation	Verification evidence	Result	Defect
NFR-P01	Database-backed multi-department dispatch must finish within two seconds locally.	TC-05; database-backed performance test	**Passed** ***(35 ms)***	–
NFR-P02	Support 100 concurrent users with responses within two seconds.	No representative concurrent-user load test	**Unverified, out of scope**	–
NFR-M01	Layered architecture and repository interfaces separate critical workflows from adapters; notification failure is isolated.	Architecture review; notification failure test	**Partially verified**	D-04

NFR-M02	Strategy and repository abstractions reduce changes to core workflow, but adding roles or department types still requires configuration and code changes.	Code and architecture review	**Partially met**	–
NFR-R01	Stored case information remains consistent across selected department projections.	TC-03, TC-05, TC-09; HTTP role tests	**Passed**	–
NFR-R02	Maintain 99.9% weekly availability.	No deployed monitoring period	**Unverified, out of scope**	–
NFR-R03	Cases, units, assignments, notifications, accounts and audit events persist across restarts.	SqlitePersistenceTests; multi-restart lifecycle	**Passed**	D-01
NFR-R04	Continue operating with 100 simultaneously open cases.	RequirementReliabilityTests.cs	**Passed**	–
NFR-S01	Selected sensitive database fields use AES-GCM; deployed TLS has not been independently verified.	Raw-database encryption and restart test	**Partially passed**	D-02
NFR-S02	Cookie authentication, middleware, role claims and scope checks restrict access.	TC-12; report access; real HTTP tests	**Passed**	D-06
NFR-S03	Passwords stored as salted PBKDF2 hashes; incorrect passwords rejected.	SecurityTests; restart authentication test	**Passed**	D-02
NFR-U01	Dispatcher, department, response-unit and prototype Admin interfaces are role-specific.	TC-12; page-model and HTTP tests	**Passed**	D-06
NFR-U02	Validation provides user-facing messages without exposing stack traces.	Invalid-input and login tests; manual review still required	**Partially verified**	D-06
NFR-U03	Users complete core tasks after no more than one hour of training.	No representative timed usability study	**Unverified**	–

2.3 Partially Met, Unverified and Out of scope Requirements

Most of the final requirements were fully verified, however, not all of them were. The following requirements were marked as not fully passed because some behavior was not completely verified, or their scope did not align within the final prototype.

Requirement	Status	Justification
FR-07	**Partially verified**	Passed automated tests for the dashboard and refresh, however the 5 second update still needs to be manually tested
NFR-P02	**Unverified**	Testing with 100 concurrent users was not done. This required load testing conditions that were outside the scope for our local prototype.
NFR-M01	**Partially verified**	Layered architecture separates critical workflows from adapters. and a regression test verifies notification-failure isolation.
NFR-M02	**Partially met**	The architecture supports extension, but adding roles or department types still requires configuration and code changes.
NFR-R02	**Unverified**	The prototype was not deployed and monitored for a week to ensure 99.9% availability.
NFR-S01	**Partially verified**	AES-GCM encryption at rest was tested, but TLS production configuration was not tested.
NFR-U02	**Partially verified**	The validation component was tested, but there was no complete manual review of the finally rendered error message output.
NFR-U03	**Unverified**	There was no user study showing that the main tasks could be carried out after only one hour of training.

3. Test Execution and Final Test Results

In the final prototype, we have added multiple changes to the prototype from the previous phase. These include priority override, nearest unit assignment, a database, improved security, and audit logging. These increased the scope of what the requirement would need to test; thus, the testing scope is increased to test these newly added features. The final testing stage is that all implementations would need to be verified on the updated functional and non-functional requirements in Task 2. These testing approaches would include unit, integration, end-to-end, security and performance.

3.1 Final Test Cases

For the final test case, we have 167 tests which all pass; the table would consist primarily of core functional and non-functional requirements. These test cases cover different test types (integration, unit, and performance) and conditions (positive, negative, invalid input, boundary and error handling).

Test ID	Test Situation	Test Type	Condition	Status
TC-01	Creating emergency cases using valid incident information.	Integration	Positive	**Pass**
TC-02	Creating emergency cases using incomplete incident information.	Integration	Invalid Input	**Pass**
TC-03	Create, prioritise, store and assign cases to relevant departments.	Integration	Positive	**Pass**
TC-04	Calculate priority using department-specific keywords.	Unit	Positive	**Pass**
TC-05	Complete in-process multi-department within 2 seconds.	Performance	Positive	**Pass**
TC-06	Skip unavailable units and assign to the closest available and compatible unit.	Integration	Negative	**Pass**
TC-07	Queue the emergency case when there are no response units available and assign it after a unit is “open”.	Integration	Negative	**Pass**

TC-08	Prevent concurrent submission from assigning the same unit to two cases.	Integration	Negative	**Pass**
TC-09	Display only relevant department cases and include new cases after refreshing,	Integration	Positive	**Pass**
TC-10	Keep a case In Progress until the final unit assigned to it signs off.	Integration	Positive	**Pass**
TC-11	Search case history using caller name, case, identifier or date.	Unit	Positive	**Pass**
TC-12	Verify role based access and reject unauthorised access	Integration (Security)	Positive & Negative	**Pass**
TC-13	Reject priority override when dispatcher does not provide a reason.	Integration	Invalid Input	**Pass**
TC-14	Verify audit logs remain ordered by checking if the newest log appears first.	Integration	Positive	**Pass**
TC-15	Verify emergency case and unit assignment information is persisted when the database restarts.	Integration	Positive	**Pass**
TC-16	Verify the password hashing uses a different salt and rejects incorrect passwords.	Unit (security)	Positive & Negative	**Pass**
TC-17	Deny coordinate location outside of the permitted boundaries.	Unit	Boundary & Invalid Input	**Pass**
TC-18	Check whether notification failure does not interrupt case creation or assignment.	Integration	Error handling	**Pass**

These 17 test cases are demonstrations of the different testing approaches used throughout all the tests rather than the entire test coverage of the system. The detailed specification of these 17 test cases is provided in **Appendix 7**.

3.2 Test Execution Results

The final tests were executed against the completed system, covering 167 test executions mentioned in task 3.1. The results are summarised below:

Test Execution Results & Metrics	Number
Planned Test	167
Executed Test	167
Passed Test	167
Failed Test	0
Blocked Test	0
Not-Run Test	0
Pass Rate:	100%

The test pass rate was calculated from the total number of passed executed tests divided by the total number of executed tests times 100. Therefore giving 100% pass rate. This shows that the current system fully satisfies the expected outcome set for all tests. *(Figure 3.2.1 shows that all 167 tests had passed in Test Explorer in Visual Studio, tested in MSTest and used SQLite for database-related tests).*

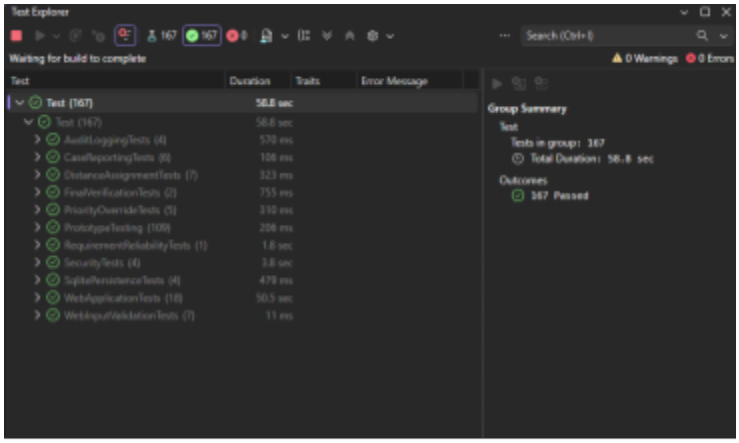


Figure 3.2.1 : All Test Case Pass Evidence

Although all 167 tests made had passed, this does not indicate that the system is fully verified for the system's full functionality and quality of the implementation defined by the requirements.

3.3 Failed Tests and Corrective Actions

For the initial test result ***(before task 3.1 Final Test Cases)***, there were 31 failed test cases out of 167 tests, resulting in a pass rate of 81.4%.

After reviewing all failed test cases, these test cases were mostly caused by the exception System.IO.IOException. These results occurred when trying to conduct testing while the cleanup process tried to delete the temporary SQLite database and locked up the files due to the connection still being used. These issues occurred throughout the different test cases such as WebApplicationTest, SqlitePersistenceTests, SecurityTests and AuditLoggingTests. Therefore, this is more related to the database connection rather than any failed testing logic implementation. These test cases would need to be fixed in order to verify that all functionalities are working as intended.

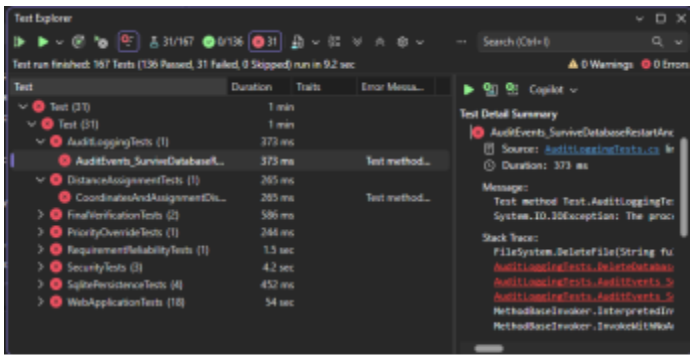


Figure 3.3.1 : System.IO.IOException

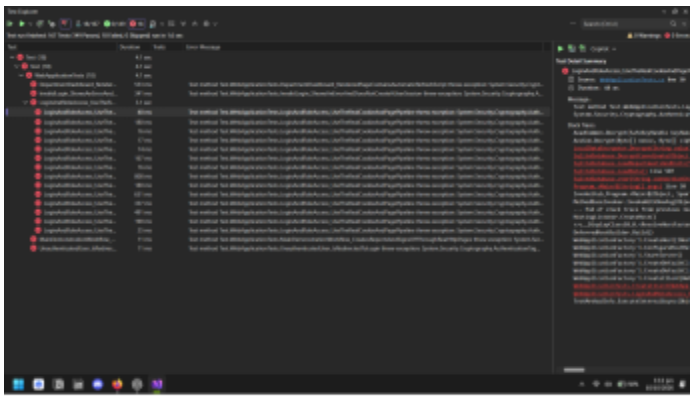


Figure 3.3.2: AES-GCM Decryption

The action taken to resolve this issue was to disable pooling for all the tests. Since each time the test requires an SQLite connection to the database, pooling could retain the database connection in the connection pool to be reused, thus preventing cleanup of the temporary database file created during the tests. Applying this change resolves the SystemIO.IOException issue.

After confirming that this update worked, this was then tested on a separate device to ensure that the issue is not related to the local environment. This resulted in 18 test cases failing rather than 31 on the initial device. These issues were related to *System.Security.Cryptography.Authentication TagMismatchException*. This occurred when the web application tried to decrypt stored data from the database.

Further investigation led to Program.cs, where this is not so much an authentication issue, but rather that the database is immediately initialised before the test could replace it with a temporary database. So the initialised database was changed so the database would only be initialised when the service requested it; this is implemented through utilising factory functions. This allowed the temporary database to be used instead of unnecessarily initialising the original database setup.

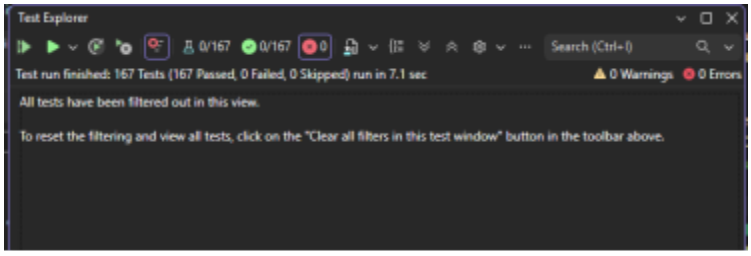


Figure 3.3.3: Result after the changes

Applying both changes towards the final testing implementation. All 167 tests passed with 0 failed tests

3.3.1 Regression testing

After applying the test changes to the initial testing described in Task 3.3 *(disabling pooling and implementing a factory function in database initialisation)*, regression testing was performed by reusing the tests applied before the updated changes. The changes could affect data persistence, case creation, authentication, and case assignment, which are critical requirements; therefore, all 167 tests were rerun instead of only the previously failed tests, as the changes made could have a wide blast radius.

After rerunning the 167 tests, the previous 31 tests that had issues with database handling were resolved, as the other tests had remained as pass. This indicates that, after the modification, no other new bugs had been found after changes made in order to solve the initial test issues.

3.5 Remaining Testing Risks

Although the 167 tests did cover major areas in the system, there are still areas where the current test still has not been verified / fully verified. These requirements that are not yet verified mainly consist of non function requirements where the current testing does not provide enough evidence. Below are critical requirements that have not been checked as well as possible future testing:

Risk	Remaining Risk / Impact	Possible further testing
NFR-P02 *(Concurrency user performance)*	The current concurrency test accounts for concurrent competing case submission but does not test for 100 simultaneous users.	Since this requirement is stated in task 2.2 that it is out of scope for this project, a test would not be created for this project, but would need to be tested when the system is further developed to be in production.
NFR-R02 *(System availability)*	This could cause system wide downtime which is important that the system is not down as the system deals with emergencies.	This is another testing that is simply out of scope for this prototype but for production, monitoring of the system over a long period of time would be required to ensure it reached 99.9% availability.
NFR-S01 *(Data transfer security)*	The test currently verifies for stored data at rest but not when the data is being transmitted.	This test is important especially when reaching production. This could be tested on HTTP / TLS and verifies that the communication is encrypted between the user and the system.
NFR-U03 *(Usability)*	Dispatcher learning curve could be slower leading to slower case assignment thus impacting units ability to quickly respond.	The test could be conducted with a group of representative dispatchers to measure the time of completion between common tasks in the system. Feedback could be received for the system usability as well.

Identify suitable test cases for automation and justify the selection checklist for automation, delete later.

- ☐ *Implement automated tests using an appropriate tool or framework where suitable*
- ☐ *Report automated test execution results using logs, screenshots, reports, or repository evidence*
- ☐ *Discuss the benefits, limitations, and maintainability of the automated test suite*
- ☐ *Where automation is not appropriate, justify the use of manual or exploratory testing*

4. Test Automation and Quality Engineering

We implemented automated tests on selected behaviour where the inputs were clear and had deterministic outputs. The priority of automated tests was given to the high-risk features that were used frequently, and to those that were vulnerable to errors during regression testing. Automated testing is important in this project because we need to keep checking the core functionality after code changes and before changes are accepted: priority calculation, unit assignment, lifecycle transitions, persistence, access control and input validation. Because these have deterministic outputs, automation supports software quality assurance by providing fast feedback and reduced manual regression effort (Dr Siddique, 2026).

4.1 Automation Selection

The areas mentioned above were selected for automated testing because their behaviours were essential to the project and were the core functionality. The following test cases received automated testing because failure in any of these areas would cause our system to fail, and they need to be tested after code changes and before changes are accepted.

Automated area	Reason for implemented automation	Evidence
Case creation and Priority	This is a core workflow that has deterministic outcomes, along with different keyword combinations	PrototypeTesting, TC-01 to TC-04
Unit assignment and waiting queue	An incorrect assignment could allocate an unavailable unit to the case or fail to queue the case.	DistanceAssignmentTests, TC-06 to TC-08
Case and unit lifecycle	The transitions between states need to be consistent from case assignment to sign-off.	TC-07, TC-10, database lifecycle tests
Database persistence	The automated tests simulate the database restarts and verify that the information is saved when the database is reopened. Easier than manual verification	SqlitePersistenceTests, FinalVerificationTests
Security and access control	Positive and negative tests should be repeated after changes are made to authentication, roles, or pages.	SecurityTests, TC-12 WebApplicationTests

Priority override and auditing	Overriding needs validation, authorisation, and all details. This has important components that should be checked repeatedly.	PriorityOverrideTests, AuditLogging Tests
Invalid and boundary input	It is important to check valid, invalid and boundary values quickly.	WebInputValidationTests, DistanceAssignmentTests
Error handling	Confirms that a notification failure will not stop the case creation or assignment from being successful.	Appendix02_NotificationFailure_DoesNotBreakCriticalDispatchWorkflow test
Reliability and performance	Need to check dispatch speed and whether the system can handle 100 open cases. Automation makes it easier for the developer, instead of having to manually check the performance.	DatabaseBackedMultiDepartmentDispatch_CompletesWithinTwoSeconds test, RequirementReliabilityTests
HTTP Web workflow	This is how a user uses the system, so it's important to always check login, access control, case creation, and case sign-off using the actual web pages.	WebApplicationTests

4.2 Automated Test Implementation

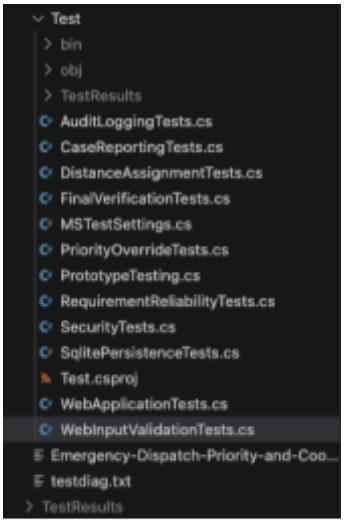


Figure 4.2.1 : Testing Directory Structure



Figure 4.2.2: AAA testing

All automated testing was implemented using MSTest 4.0.2, which was written in C# using .NET 10. We used MSTest because it integrated our project solution easily and required little effort during setup. As shown in Figure 4.2.1, the automated tests were separated by the functionality or quality area that was being tested. For example, DistanceAssignmentTests.cs contains code for testing the assignment of response units using their distance, and SqlitePersistenceTests.cs contains code for testing overall database persistence.

All of the tests used AAA testing as shown in Figure 4.2.2. Arrange creates the case, unit, repository, etc. Act performs the action/behaviour that is being tested. Assert compares the actual result with the expected result. Unit testing was used to check the individual components and how they worked in isolation.

This included the priority keyword calculation, coordinate validation, and input boundary. Unit tests are fast because they only test small pieces of logic instead of using the whole web application.

4.3 Automated Test Results

show the automated run result, probably 167/167 passed, with Test Explorer/log screenshot evidence. Don't repeat all of Section 3; just focus on the automation result.

```
Test summary: total: 167, failed: 0, succeeded: 167, skipped: 0, duration: 2.4s
Build succeeded in 3.0s
fatehbhular@Fatehs-MacBook-Pro Emergency-Dispatch-Priority-and-Coordination-System %
```

Figure 4.3.1 : Automated Testing Results

The final testing suite produced 167 automated MSTest executions where all 167 tests passed, including all data-driven rows *(Figure 4.3.1)*. There were no failed, blocked, or skipped tests because there was an acceptable amount that could be run in a short period of time. These tests include unit, integration, database, security, reliability, performance, and HTTP behaviour. While this result shows that all of our expected outcomes were satisfied, it doesn't mean that our system is defect free. Because not everything was tested, as shown in section 3.5, defects relating to performance, usability, and accessibility could still remain in the system.

4.4 Automation Maintainability

When creating the tests, we sorted them into different files for easier maintenance and organisation. Figure 4.2.1 shows how there were tests for different behaviours of the system like audit logging, priority override, web application, etc. All tests used the Arrange, Act, and Assert template, shown in figure 4.4.1, which is commonly used to write well defined tests

```
public sealed class CaseReportingTests
{
    [TestMethod]
    0 references
    public void DispatcherReport_LoadsCaseAndOrdersItsAuditTimeline()
    {
        // Arrange
        var cases = new InMemoryCaseRepository();
        var audit = new InMemoryAuditRepository();
        var dispatchCase = AssignedMedicalCase();
        cases.Add(dispatchCase);
        audit.Add(Event(dispatchCase, AuditEventTypes.UnitAssigned, 2));
        audit.Add(Event(dispatchCase, AuditEventTypes.CaseCreated, 1));
        var page = CreatePage(cases, audit, DispatchAccount());

        // Act
        var result = page.OnGet(dispatchCase.Id);

        // Assert
        Assert.IsInstanceOfType<PageResult>(result);
        Assert.AreSame(dispatchCase, page.DispatchCase);
        Assert.IsTrue(page.CanViewPrivateDetails);
        CollectionAssert.AreEqual(
            new[] { AuditEventTypes.CaseCreated, AuditEventTypes.UnitAssigned },
            page.AuditEvents.Select(item => item.EventType).ToArray());
    }
```

Figure 4.4.1 : Arrange, Act, Assert structure

in code. The test names describe the input or situation and what the expected result of the test is. This makes it it easier for any developers looking through the test files, and helps with understanding what tests are being performed, making the test suite maintainable.

```
2 references
private static AuditEvent Event(Case dispatchCase, string eventType, int minute) =>
    new(Guid.NewGuid(), new DateTimeOffset(2026, 10, 7, 1, minute, 0, TimeSpan.Zero), eventType,
        "dispatch01", dispatchCase.Id, dispatchCase.CaseNumber, null, null, null, null);

2 references
private static DemoAccount DispatchAccount() =>
    new("dispatch01", "Dispatcher", DemoRoles.Dispatcher, null);
```

Figure 4.4.2 : Helper methods for testing

For one section of tests, some tests would use identical setup steps or functions. To make the test suite more maintainable, helper methods were implemented to create services, databases, and input objects. This not only reduced the total number of code, but also made it easier to create new tests that need to use the helper method.

```
6 references
private static (SqliteCaseRepository Cases, SqliteDepartmentRepository Departments) CreateStore(string path)
{
    var database = new SqliteDatabase($"Data Source={path};Pooling=False");
    return (new SqliteCaseRepository(database), new SqliteDepartmentRepository(database));
}
```

Figure 4.4.2 : Helper methods for testing

Database tests were separated from the rest of the tests for test isolation. Each test was given their own temporary SQLite database and files, so that they wouldn't be added to the database used for normal demonstration. After each test was executed, the database along with the files were deleted.

4.5 Manual and Exploratory Testing

explain what automation doesn't prove. Good examples from your own report are the actual visible five-second dashboard refresh, fully rendered validation/error messages, and representative usability testing. These are already identified as evidence gaps elsewhere, so this will stay consistent.

5. Continuous Integration and DevOps QA

5.1 Development and Integration Workflow

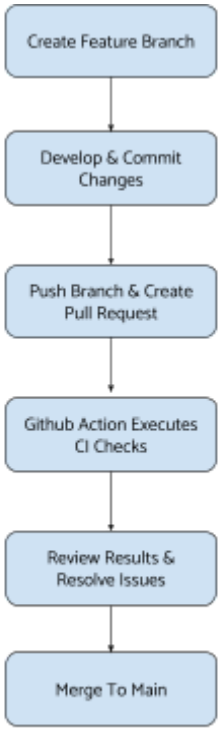


Figure 5.1.1: Development and Integration Workflow

Throughout the development workflow, github version control was used for this system. Each functionality is developed through different branches outside of the main branch. Commits were made to each individual branch when the feature was successfully developed in the branch. The separate branches allows each developer to work independently without impacting the main branch though conflict would still occur during integration to the main branch. Then a pull request is created where to review the changes made and merge conflicts are checked to ensure that there are no conflicting codes and resolve any issues found.

Continuous Integration workflow is included by the use of Github Actions which allows the automatic workflow to be applied whenever there are changes pushed towards the testing branch or pull request with the main branch. In this automatic workflow, the system is built and automatic tests are run to make sure the behavior of the systems passed the set tests. If

the test or build failed developers would review the changes before allowing the merging with the main branch.

5.2 Continuous Integration Pipeline

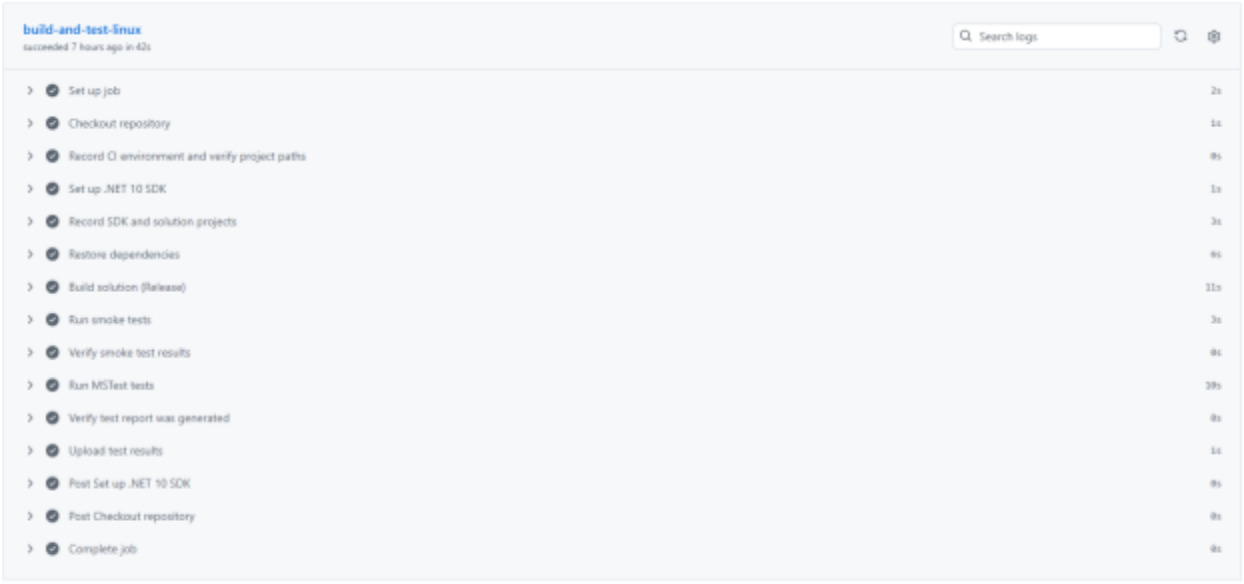


Figure 5.2.1: Successful Github Action CI Pipeline Execution

The continuous pipeline was configured in the file .github/workflows/ci.yml using Ubuntu linux runner with the .NET SDK. The pipeline is triggered when there is a pull request to the main branch on updates and creations or towards the Testing Branch for CI. Once this is triggered the pipeline goes through a series of steps. It first ensures that the path and environment are correct inside the repository. This is followed by dependencies installing and building. This checks whether the emergency dispatch system complies properly and successfully.

After confirming the build is successful, The smoke test was utilized (The core functionality tests would be stated in task 5.4) to check the core functionality of the system. If the smoke test passes, the rest of the automated test would run. If all tests are passed, the test results are stored in a TRX file for reviewing and checking failures if there is.

5.3 Quality Gates

Quality Gates are used to stop code from other branches from being incorporated into the main branch without meeting certain criteria, using automated checkpoints to ensure that each point the updated code reaches meets a set standard. Implementing quality gates improves the Quality test in many ways, including having a baseline quality check and detecting defects earlier. However, this would still need human intervention to check commits that failed to meet the checkpoint standard. Below are the set criteria that the produced code needs to meet before reaching integration:

Quality Gate	Passing Criteria	Consequences	Justification
Environment Check	The runner is using linux, and the solution has to be .slnx and the test.csproj file exists.	CI fails, restore dependency, verify build, smoke test, full automated test and produce test report are skipped.	It's important before the other checks, since it validates that the project is properly configured.
Restore Dependency	Check that the project can restore dependencies before building.	CI fails, verify build, smoke test, full automated test, and produce test report are skipped.	Ensures that the dependencies required to build and test are available.
Verify Build	The system build correctly compiles.	CI fails, smoke test, full automated test, and produce test report are skipped.	Make sure that the system compiles without errors, this allows it to be detected earlier to prevent unnecessary test execution.
Smoke Test	All 89 core functionality tests passed.	CI fails, full automated tests, and produce test reports are skipped.	This is to make sure that the main emergency dispatch functionality fully passes the test and that it acts as a

			regression test to ensure that at the very least the main functionality of the system is operational and unchanged.
Full Automated Test	All 167 automated testing passed.	CI fails, report is skipped.	A form of regression test to check if any introduced problems affected the existing functionality covered by the automated tests.
Produce Test Report	The full test report is produced and saved.	CI fails, because the test file is missing or empty.	The report is used for developers to analyze changes and review issues.

These quality gates are arranged sequentially from the layout of the table, meaning that the failure of a quality gate means the quality gates below would be skipped.

5.4 Smoke testing

Test Case	**Core Functionality**	**Purpose**
TC-01	Create an emergency case using valid information	Check that the emergency cases can be recorded successfully.
TC-03	Route a case to the correct department	Verify that the department receives the correct case.
TC-04	Calculate the emergency case priority	Verify the case incident receives the appropriate priority.
TC-06	Assign an available response unit	Check that available and compatible units are assigned properly.

TC-10	Complete a case after unit sign-off	Check that units were able to sign off and the corresponding case is “Closed”.

The smoke tests were selected from the identification of core functionalities in task 1.4, where these tests represent the main emergency dispatch workflow, TC-01, TC-03, TC-04, TC-06 and TC-10. Any failures of these functionalities would interrupt the main dispatch process.

There are 89 recorded tests in the smoke tests stage, although there are 5 core functional test cases that were selected, this is because TC-04 which uses DynamidData to execute 85 different input combinations to check that the priority calculation rules all pass. The other 4 test cases each have 1 test which makes 89 tests in total. In the successful CI run in the testing branch, 89 executions were passed with 0 failures.



Figure 5.4.1: Example of Smoke Test Quality Gate Results

5.5 CI Evidence and Limitations



Figure 5.5.1: Example of Total Automated Tests in Test Quality Gate Results

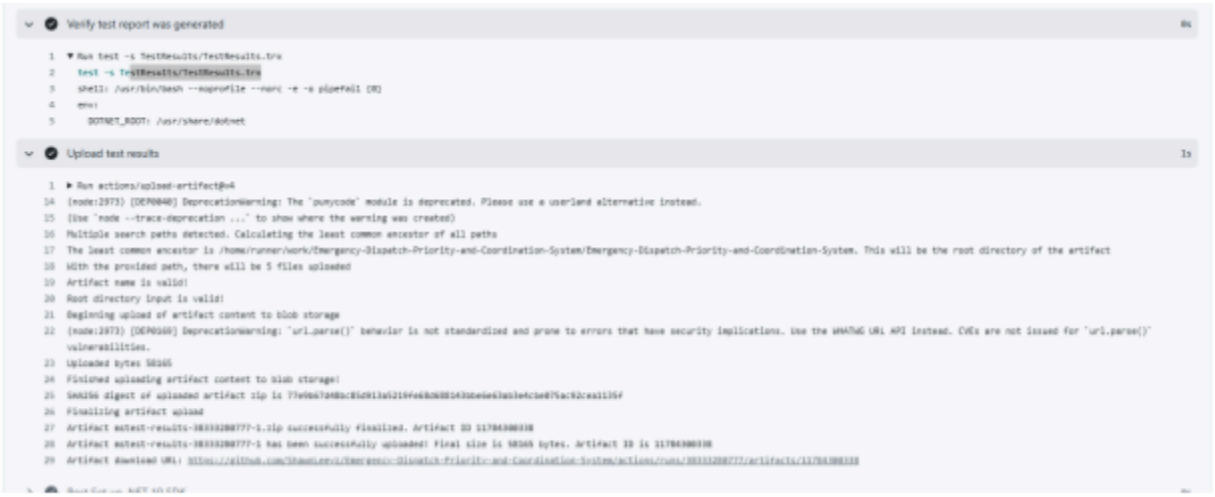


Figure 5.5.2: Example of Test Report Created & Uploaded

Figure 5.5.1 shows an example of all 167 automated tests passing in GitHub Actions. It ensures that modifications to the code do not affect functionality that was tested in the automated tests by rerunning all the automated tests. Figure 5.5.2 shows an example of the report being generated and uploaded as an artifact in GitHub Actions, so that issues could be investigated through analyzing the produced test report.

The implemented quality gates are an improvement to the development process. The automatic pipeline helps check whether the code builds properly, check dependencies, perform testing, and generate test reports are important. However, further quality gate implementation in the pipeline would be required to improve the quality of the code that is merged into the main branch. The pipeline lacks a measurable way of code coverage. Even though there are 167 automated tests, it does not mean that most of the systems have been tested. Additionally, there is no vulnerability checking of the system;

there are security checks, but there are no security checks for dependencies, etc. Further, there are no protections for merging code to the main branch even if CI did not pass.

6. Defect Management and Root Cause Analysis

6.1 Defect Management Framework

Defects and quality issues were identified throughout development, code review, and testing. We documented major defects in the defect register and assigned them a severity, which is their impact on the application, and also a priority, which is how quickly it should be resolved. Every time a defect is detected and confirmed, its root cause is analysed, the appropriate action is taken, and then finally a regression test is done before closing the defect.

Below is the table that defines our severity and priority categories;

Category	Level	Definition
Severity	Critical	Prevents safe use, results in loss of significant data, or results in unauthorised access to significant data
Severity	High	Breaks a major system function
Severity	Medium	Impacts a non-critical workflow or reduces the quality of a verification criterion
Severity	Low	Minor issue, or graphical issue
Priority	P1	Needs to be fixed immediately
Priority	P2	Must be fixed before final release
Priority	P3	Future improvement
Priority	P4	Optional improvement

The defect statuses used were New, Confirmed, In Progress, Fixed, Retested, Closed, and Deferred.

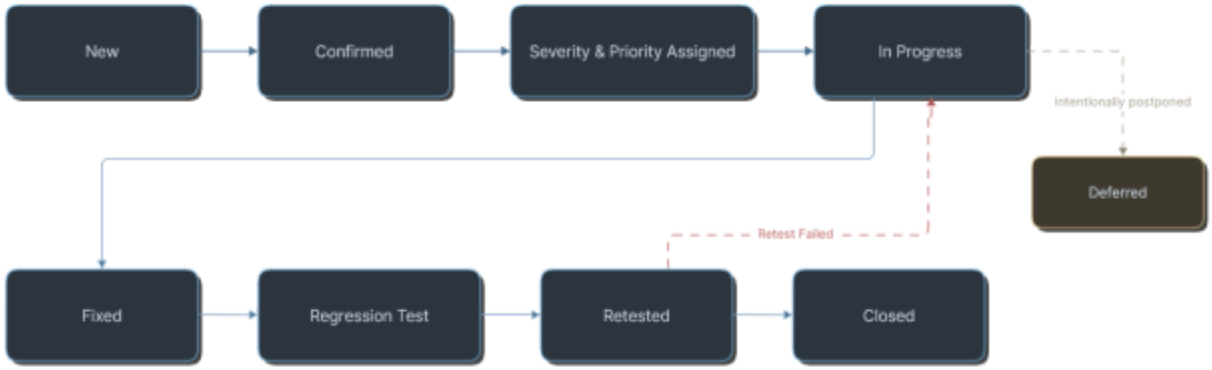


Figure 6.1.1 : The defect lifecycle used in the project

The status of each defect was tracked throughout its lifecycle, as shown in Figure 6.1. Once a defect is detected and confirmed, it is assigned a severity and priority before the defect has been analysed and the correct solution has been implemented. After a solution is proposed, a regression test is done, and then the defect is retested once again. If the retest is successful, the status of the defect will change to 'Closed', and if the retest fails, it is returned to 'Progress' for further analysis. For defects held in Progress', if they are intentionally postponed, they can be moved to 'Deferred'.

6.2 Final Defect Register

The final defect register contains the major defects and quality issues that were identified during the development of the final prototype. All the defects found were corrected, retested, and closed before the final prototype was completed.

ID	Defect / Quality Gap	Severity / Priority	Fix	Status
D-01	Data was lost after restarting the application.	High / P1	Switched from in memory storage to SQLite.	Closed

D-02	Credentials and sensitive data could be readable in storage.	High / P1	Added PBKDF2 password hashing and AES-GCM encryption.	Closed
D-03	The first available unit was selected instead of the nearest unit.	High / P2	Added coordinate distance assignment	Closed
D-04	Notification failure could cause a successful dispatch to appear to fail	High / P1	Isolated notification failure from the critical dispatch workflow.	Closed
D-05	HTTP tests could use the developer database.	Medium / P1	Replaced with an isolated temporary test database.	Closed
D-06	Earlier tests lacked negative and real workflow coverage.	Medium / P2	Added more negative, boundary, restart, audit, and HTTP testing	Closed

The root causes, corrective actions, regression evidence and preventive measures for the selected significant defects are analysed in Section 6.3.

6.3 Root Cause Analysis of Significant Defects

The following three defects were selected for further root cause analysis, these being D-01, D-04 and D-05. These defects as shown from table in section 6.2 are related to system reliability, fault handling and testing quality. The five whys technique was used because each of the defects had a relatively clear sequence of events that we can follow in order to find the root cause of the high level problem. In this case the fishbone diagram was not suitable for this cause because the defects were not complex enough to require an analysis across a large number of possible causes. In this case, the five whys technique was much more straightforward to determine the root cause, and the possible preventive actions.

6.3.1 D-01 *(Restart Data Loss)*

Problem: Cases, assignments and unit changes were lost after the application restarts

Impact: The application is not able to reliably preserve important emergency case information

5 Why's

1. ***Why was data lost?*** Because the app stored data only in memory.
2. ***Why did the app only store data in memory?*** Because the original prototype used in memory repositories.
3. ***Why was this still a problem in the final version?*** Because the final system needed data to persist after a restart, but the storage design had not originally supported that.
4. ***Why was this not found earlier?*** Because the previous tests reused repository objects rather than testing an actual restart with a durable database.
5. ***Why did the previous tests do that?*** Because they were built from the original in-memory prototype rather than the restart recovery.

Root cause: Original in-memory storage design was carried on from the prototype, and earlier tests also were built according to this design, so this was not properly tested.

Corrective Action: Replace in-memory repositories with the SQLite-backed ones so that data is persistent after a restart.

Preventive Action: Add restart testing to make sure stored data is not lost when the application is closed and reopened.

Verification: SqlitePersistenceTests and FullDatabaseLifecycle_RemainsCorrectAcrossMultipleRestarts verify that the stored data remains available after reopening the database.

6.3.2 D-04 *(Notification Failure Boundary)*

Problem: A notification failure could cause a successful dispatch to look like it had failed.

Impact: Even though it indicated an error, the case and unit assignment could have already been saved.

5 Why's

1. ***Why did the dispatch appear to fail?*** Because the notification service threw an error back through the dispatch process.
2. ***Why was the error allowed to pass through the dispatch process?*** Because there was no failure boundary around the notification service.
3. ***Why was there no failure boundary?*** Because notifications were handled as part of the same workflow as case storage and unit assignment.
4. ***Why were they handled as part of the same workflow?*** Because the design did not separate critical operations from non-critical operations.
5. ***Why were critical and non-critical operations not separated?*** Because their different failure behaviour had not been considered in the original workflow design.

Root cause: The workflow did not separate non-critical notification failures from critical dispatch operations

Corrective Action: Notification failures were isolated so they could no longer cause an already successful dispatch to appear unsuccessful.

Preventive Action: Separate critical and non-critical operations during design and include failure tests for non-critical components.

Verification: Appendix02_NotificationFailure_DoesNotBreakCriticalDispatchWorkflow uses a failing notifier and verifies that the case is still stored and assigned.

6.3.3 D-05 *(HTTP Test Database Isolation)*

Problem: HTTP tests are able to use the developer database instead of their own isolated test database.

Impact: Tests could be reading and writing development data, making the test results more unreliable.

5 Why's

1. ***Why could the HTTP tests be using the developer database?*** The test server was still using the application's Sqlite Database.
2. ***Why was it still using the application's database?*** We changed the test connection setting, but this didn't replace the existing database service.
3. ***Why didn't changing the connection setting replace it?*** The application had already registered the Sqlite Database before the service even started.
4. ***Why was the registered database not replaced?*** Because the test setup assumed changing the configuration was enough to use the temporary database.
5. ***Why was this problem not identified sooner?*** The HTTP tests mostly read data, so there weren't any clear signs we were using the wrong database.

Root cause: Our test setup changed the database configuration but didn't replace the existing SqliteDatabase service.

Corrective Action: HTTP test setup removed the existing database service and replaced it with a temporary database for the test workflow.

Preventive Action: HTTP and other integration tests use a separate temporary database which can't access development data.

Verification: WebApplicationTests now replaces SqliteDatabase with a temporary database used only for the test workflow.

6.4 Defect Prevention and Quality Improvement

7. Quality Testing

- [ ] *Conduct quality testing relevant to the project (security, performance, usability, accessibility, compatibility, reliability, or maintainability)*
- [ ] *Address at least 2 quality areas where applicable*
- [ ] *Use risk-based reasoning to justify the selected testing activities*
- [ ] *Document testing evidence, results, limitations, and recommended improvements*

7.1 Risk-Based Selection of Quality Areas

7.2 Quality Area One: [e.g. Security]

7.3 Quality Area Two: [e.g. Performance]

7.4 Results and Limitations

7.5 Recommended Improvements

8. Quality Metrics and Final Release Decision - lao

- [ ] *Present relevant quality metrics (test pass rate, defect severity distribution, requirement coverage, test coverage, automation coverage, code quality indicators, commit activity)*
- [ ] *Evaluate the completed system against requirements, acceptance criteria, and quality attributes*
- [ ] *Summarise testing results, defect status, metrics, limitations, and residual risks*
- [ ] *State whether the system is ready for release, conditionally ready, or not ready for release*
- [ ] *Justify the release decision using evidence*

8.1 Final Quality Metrics

8.2 Requirements and Acceptance-Criteria Evaluation

8.3 Outstanding Defects and Residual Risks

8.4 Final Release Decision

9. AI-Assisted Development and Critical Reflection

- [ ] *Explain how GitHub Copilot was used (implementation, testing, debugging, refactoring, or QA activities)*
- [ ] *Provide representative evidence of AI-assisted development*
- [ ] *Explain how AI-assisted outputs were reviewed, modified, tested, accepted, or rejected*
- [ ] *Identify quality risks found in AI-assisted outputs (incorrect logic, insecure code, poor maintainability, unsuitable assumptions)*
- [ ] *Reflect on how AI-assisted development affected productivity, learning, testing, software quality, and professional practice*

9.1 AI-Assisted Activities

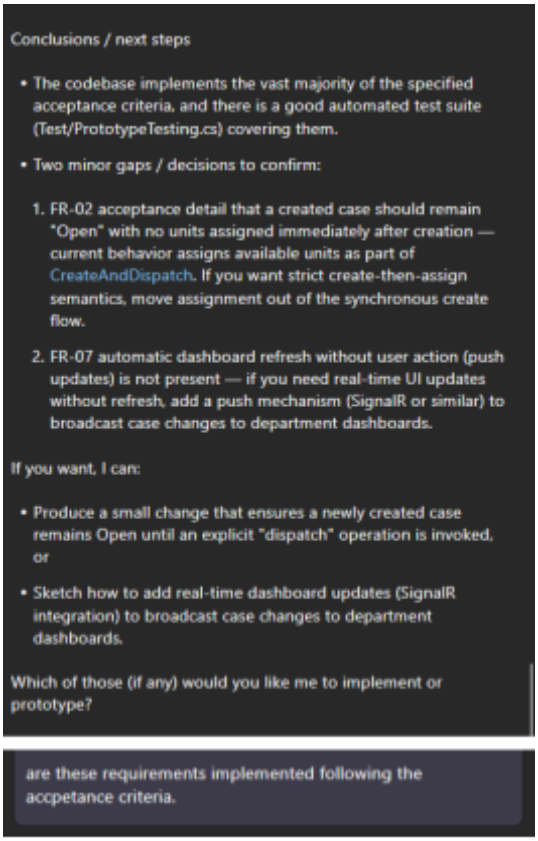


Figure 1.2.1: Github Copilot Evaluation

In the previous assignment, the functional requirements and non-functional requirements are defined and the acceptance criteria are created to describe expected behaviour of the core functional requirements. GitHub Copilot has been used to review the requirements that are gaps identified of core functionality with acceptance criteria. FR-02 and FR-07 are flagged as gaps that need to be considered for further changes. FR-01, FR-04, FR-05, FR-06, FR-08, FR-09 and FR-11 are not found as issues and are considered potential issues by GitHub Copilot. It identified that FR-02 acceptance criteria is not fully met, this criteria in FR-02 “Given a case is newly created from a recorded call, when creation completes, then the case status is set to “Open” with no units assigned to it yet”. FR-07 is flagged because GitHub described that the dashboard does not automatically refresh without user action, which doesn’t meet this: “Given a department’s dashboard is open, when the information of a case changes e.g. priority, location, then the dashboard should refresh and show the new changes automatically.” and “Given a department’s dashboard is open, when a case is routed to that department, then the dashboard should display the new case automatically.” When personally

investigating whether the FR-02 implementation meets the acceptance criteria gaps during testing, there isn't an issue because if there is a unit available, then the case would be assigned, thus receiving the status of "InProgress". The acceptance criteria is only met when there are no available units in that relevant department. Therefore, the acceptance criteria would need to be updated that the criteria is only for when there are no available units. For FR-07, after personal testing of the prototype, it has met its expected behaviour since the dashboard is updated without manually refreshing. The suggested GitHub flagged issues because it assumed that automatic updates mean adding a push mechanism (SignalR) to satisfy the stated criteria.

9.2 Representative GitHub Copilot Evidence

9.3 Human Review and Validation

9.4 AI-Related Quality Risks

9.5 Critical Reflection

10. Quality Improvement and Future Recommendations

- [ ] *Identify the main quality lessons learned from the project*
- [ ] *Recommend improvements to the development and QA process*
- [ ] *Propose future improvements to testing, automation, CI/CD, monitoring, security, usability, accessibility, or maintainability*
- [ ] *Explain how the project could evolve toward stronger continuous quality engineering practice*

10.1 Key Quality Lessons - Leo

10.2 Improvements to the Development and QA Process - fateh

10.3 Future Testing and Automation Improvements - Leo

10.4 Future Continuous Quality Engineering - Shawn

11. Team Contribution and Professional Reflection

- [ ] *Provide a contribution statement for each group member*
- [ ] *Provide evidence from GitHub commits, project artefacts, testing work, documentation, and implementation tasks*
- [ ] *Include a brief individual reflection from each group member*

11.1 Contribution Statement

11.2 GitHub and Project Evidence

11.3 Individual Reflection – Member One

11.4 Individual Reflection – Member Two

11.5 Individual Reflection – Member Three

Delivarables (Notes)

Prepare a 15-slide presentation covering: title/members/problem/stakeholders/scope, key requirements, final prototype and changes since Assessment 1, testing approach/coverage/results, automation/CI/CD, defects and RCA, quality metrics and release decision, Copilot use and limitations, individual contributions and lessons learned
All students participate in the demonstration and show the working prototype
Demo includes key features, important test cases, automated tests/CI results, defect fixes, evidence major requirements are satisfied
Be prepared to answer questions on the project, code, testing, quality artefacts, AI-assisted development, and individual contributions

Deliverables

- Final Project and Software Quality Assurance Report: 6,000–7,000 words
- Source code and GitHub repository link, with evidence from GitHub commits
- Contribution statement for each group member
- Project presentation
- Turnitin Similarity Report

References:

O Aotearoa, R. I. T. R. I. (n.d.). Pressure grew to fix the 111 emergency call system, with the 1958 model still in use. RNZ. https://www.rnz.co.nz/news/emergencies_local/585202/pressure-grows-to-fix-111-emergency-call-system-with-1958-model-still-in-use

1News Reporters. (2026, August 10). St John bracing for another 111 spike amid record demand. 1News. https://www.1news.co.nz/2026/08/10/st-john-bracing-for-another-111-spike-amid-record-demand/

GeeksforGeeks. (2026, May 13). Repository design pattern. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/repository-design-pattern/

GeeksforGeeks. (2026, May 14). Strategy design pattern. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/

Siddique, A. (2026, September 14). *Test automation: Quality engineering foundations* [Lecture slides]. Auckland University of Technology.